REACT NATIVE

React Native 6 — Recipe Finder

Пошаговое учебное руководство
от условия до проверки и сдачи

Обязательная работа

Учебный комплект • 2026

Что требуется

- Ввести ингредиент, запросить TheMealDB Filter API `filter.php?i=` и показать name/thumbnail в FlatList.
- Использовать TypeScript/Expo SDK 54, strict compiler, typed tests и entry point.
- Обработать empty input, loading, empty result, HTTP/network/malformed API error и Retry.
- Внешний payload принимать как `unknown`, проверять runtime guards и внедрять `fetchImpl`.

Что получится

Приложение с пятью UI state, безопасным преобразованием TheMealDB в `Meal[]` и offline Jest tests без ключа.

Подготовка на macOS

Нужны Node.js 20.19.4+, npm и Expo Go/simulator. `npm install` выбирает/обновляет зависимости и lock; в готовом проекте `npm ci` ставит lock точно.

Создание с нуля

КОД · BASH

```
npx create-expo-app@latest 06-recipe-finder --template expo-template-blank-typescript@sdk-54 --no-agents-md
cd 06-recipe-finder
npx expo install react-dom react-native-web
npm install --save-dev typescript@~5.9.2 @types/react@~19.1.0 @types/jest@^29.5.14 jest@29.7.0 jest-expo@~54.0.18 @testing-library/react-native@13.3.3 react-test-renderer@19.1.0
```

Структура файлов

КОД · TEXT

```
06-recipe-finder/
├── App.tsx
├── index.ts
├── tsconfig.json
├── package.json
├── src/meals.ts
└── __tests__/{meals.test.ts,App.test.tsx}
```

Lock, node_modules и dist-web генерируются, поэтому не печатаются.

Пошаговая реализация

package.json

КОД · JSON

```
{
  "name": "fullstack-homework-recipe-finder",
  "version": "1.0.0",
```

```
  "main": "index.ts",
  "scripts": {
    "start": "expo start",
    "android": "expo start --android",
    "ios": "expo start --ios",
    "web": "expo start --web",
    "test": "jest --watchAll=false",
    "typecheck": "tsc --noEmit"
  },
  "dependencies": {
    "expo": "~54.0.36",
    "expo-status-bar": "~3.0.9",
    "react": "19.1.0",
    "react-native": "0.81.5",
    "react-dom": "19.1.0",
    "react-native-web": "^0.21.0"
  },
  "devDependencies": {
    "@testing-library/react-native": "13.3.3",
    "jest": "29.7.0",
    "jest-expo": "~54.0.18",
    "react-test-renderer": "19.1.0",
    "@types/jest": "^29.5.14",
    "@types/react": "~19.1.0",
    "typescript": "~5.9.2"
  },
  "jest": {
    "preset": "jest-expo"
  },
  "private": true
}
```

`main` и `typecheck` переводят entry/compiler на TS; web dependencies нужны Expo export.

tsconfig.json

КОД · JSON

```
{
  "extends": "expo/tsconfig.base",
  "compilerOptions": {
    "strict": true,
    "noEmit": true
  }
}
```

Expo base настраивает JSX, strict запрещает неявный any, noEmit не дублирует Metro.

src/meals.ts

КОД · TS

```
export type Meal = {
  id: string;
  name: string;
  thumbnail: string;
};

export type IngredientResult =
  | { ok: true; ingredient: string }
  | { ok: false; error: string };

export type MealResult =
  | { ok: true; meals: Meal[] }
  | { ok: false; error: string };
```

```
export type FetchResponse = {
  ok: boolean;
  json: () => Promise<unknown>;
};

export type FetchImplementation = (url: string) => Promise<FetchResponse>;

const requestError = 'Could not load recipes. Try again.';

function isRecord(value: unknown): value is Record<string, unknown> {
  return typeof value === 'object' && value !== null;
}

function isMealPayload(value: unknown): value is {
  idMeal: string;
  strMeal: string;
  strMealThumb: string;
} {
  return (
    isRecord(value) &&
    typeof value.idMeal === 'string' &&
    typeof value.strMeal === 'string' &&
    typeof value.strMealThumb === 'string'
  );
}

export function normalizeIngredient(text: string): IngredientResult {
  const ingredient = text.trim().toLowerCase();
  return ingredient === ''
    ? { ok: false, error: 'Enter one ingredient.' }
    : { ok: true, ingredient };
}

export function buildMealsUrl(ingredient: string): string {
  return `https://www.themealdb.com/api/json/v1/1/filter.php?i=${encodeURIComponent(ingredient)}`;
}

export function normalizeMealResponse(payload: unknown): MealResult {
  if (!isRecord(payload) || !Object.hasOwn(payload, 'meals')) {
    return { ok: false, error: requestError };
  }

  const { meals } = payload;
  if (meals === null) {
    return { ok: true, meals: [] };
  }
  if (!Array.isArray(meals) || !meals.every(isMealPayload)) {
    return { ok: false, error: requestError };
  }

  return {
    ok: true,
    meals: meals.map((meal) => ({
      id: meal.idMeal,
      name: meal.strMeal,
      thumbnail: meal.strMealThumb,
    })),
  };
}

export async function fetchMeals(
  ingredient: string,
  fetchImpl: FetchImplementation = fetch,
): Promise<MealResult> {
  try {
    const response = await fetchImpl(buildMealsUrl(ingredient));
```

```
    if (!response.ok) {
      return { ok: false, error: requestError };
    }

    return normalizeMealResponse(await response.json());
  } catch {
    return { ok: false, error: requestError };
  }
}
```

`Meal` — локальная модель. `ok` unions безопасно открывают success/error fields; minimal Fetch contract делает fake простым. Guards доказывают unknown payload до доступа. `null` — valid empty branch, malformed/HTTP/catch — retryable error; URL кодирует ingredient.

App.tsx

КОД · TSX

```tsx
import { useState } from 'react';
import { Button, FlatList, Image, Text, TextInput, View } from 'react-native';
import {
  fetchMeals,
  normalizeIngredient,
  type FetchImplementation,
  type Meal,
} from './src/meals';

type AppProps = {
  fetchImpl?: FetchImplementation;
};

export default function App({ fetchImpl }: AppProps): React.JSX.Element {
  const [ingredientText, setIngredientText] = useState('');
  const [meals, setMeals] = useState<Meal[]>([]);
  const [loading, setLoading] = useState(false);
  const [errorMessage, setErrorMessage] = useState('');
  const [searched, setSearched] = useState(false);

  async function search(): Promise<void> {
    const ingredient = normalizeIngredient(ingredientText);
    if (!ingredient.ok) {
      setErrorMessage(ingredient.error);
      return;
    }

    setMeals([]);
    setLoading(true);
    setErrorMessage('');
    const result = await fetchMeals(ingredient.ingredient, fetchImpl);
    setLoading(false);
    setSearched(true);

    if (!result.ok) {
      setErrorMessage(result.error);
      return;
    }

    setMeals(result.meals);
  }

  function renderMeal({ item }: { item: Meal }): React.JSX.Element {
    return (
      <View>
        <Image
          accessible
```

```
          accessibilityRole="image"
          accessibilityLabel={`Photo of ${item.name}`}
          source={{ uri: item.thumbnail }}
          style={{ width: 80, height: 80 }}
        />
        <Text>{item.name}</Text>
      </View>
    );
  }

  return (
    <View style={{ gap: 12, padding: 16 }}>
      <Text>Ingredient</Text>
      <TextInput
        accessibilityLabel="Ingredient"
        placeholder="e.g. chicken"
        value={ingredientText}
        onChangeText={setIngredientText}
      />
      <Button title="SEARCH RECIPES" disabled={loading} onPress={search} />
      {loading ? <Text accessibilityLiveRegion="polite">Loading recipes…</Text> : null}
      {errorMessage ? (
        <Text
          accessibilityLabel="Recipe error"
          accessibilityRole="alert"
          accessibilityLiveRegion="assertive"
        >
          {errorMessage}
        </Text>
      ) : null}
      {searched && !loading && !errorMessage && meals.length === 0 ? (
        <Text accessibilityLabel="Recipe empty" accessibilityLiveRegion="polite">
          No recipes found.
        </Text>
      ) : null}
      {errorMessage === 'Could not load recipes. Try again.' ? (
        <Button title="RETRY SEARCH" disabled={loading} onPress={search} />
      ) : null}
      <FlatList data={meals} keyExtractor={(item) => item.id} renderItem={renderMeal} />
    </View>
  );
}
```

Optional fetchImpl maintains real default and enables deterministic tests. Five hooks distinguish input/result/loading/error/first search. `search` validates before fetch, clears stale meals, narrows result and presents each live-region/error branch. `renderMeal` types props and Image accessibility.

index.ts

КОД · TS

```
import { registerRootComponent } from 'expo';
import App from './App';

registerRootComponent(App);
```

Expo registers the same App for native/web.

__tests__/meals.test.ts

КОД · TS

```
import {
  buildMealsUrl,
```

```
  fetchMeals,
  normalizeIngredient,
  normalizeMealResponse,
  type FetchImplementation,
  type FetchResponse,
} from '../src/meals';

const mealPayload = {
  meals: [
    {
      idMeal: '52772',
      strMeal: 'Teriyaki Chicken',
      strMealThumb: 'https://img/chicken.jpg',
    },
  ],
};

function createResponse(payload: unknown, ok = true): FetchResponse {
  return { ok, json: async () => payload };
}

test('normalizes an ingredient and encodes it in the documented MealDB URL', () => {
  expect(normalizeIngredient('  chicken breast ')).toEqual({ ok: true, ingredient: 'chicken
breast' });
  expect(buildMealsUrl('chicken breast')).toBe(
    'https://www.themealdb.com/api/json/v1/1/filter.php?i=chicken%20breast',
  );
});

test('normalizes a valid MealDB payload into application meals', () => {
  expect(normalizeMealResponse(mealPayload)).toEqual({
    ok: true,
    meals: [{ id: '52772', name: 'Teriyaki Chicken', thumbnail: 'https://img/chicken.jpg' }],
  });
});

test.each<unknown>([
  null,
  {},
  { meals: 'not an array' },
  { meals: [{ idMeal: '52772', strMeal: 'Incomplete meal' }] },
])('rejects a malformed external payload: %p', (payload) => {
  expect(normalizeMealResponse(payload)).toEqual({ ok: false, error: 'Could not load recipes. Try
again.' });
});

test('fetchMeals returns normalized list data without network access', async () => {
  const fetchImpl: FetchImplementation = jest
    .fn<Promise<FetchResponse>, [string]>()
    .mockResolvedValue(createResponse(mealPayload));

  await expect(fetchMeals('chicken', fetchImpl)).resolves.toEqual({
    ok: true,
    meals: [{ id: '52772', name: 'Teriyaki Chicken', thumbnail: 'https://img/chicken.jpg' }],
  });
  expect(fetchImpl).toHaveBeenCalledWith(
    'https://www.themealdb.com/api/json/v1/1/filter.php?i=chicken',
  );
});

test.each<[FetchImplementation]>([
  [jest.fn<Promise<FetchResponse>, [string]>().mockResolvedValue(createResponse({}, false))],
  [jest.fn<Promise<FetchResponse>, [string]>().mockRejectedValue(new Error('offline'))],
])('fetchMeals exposes a retryable request error', async (fetchImpl) => {
  await expect(fetchMeals('chicken', fetchImpl)).resolves.toEqual({
    ok: false,
    error: 'Could not load recipes. Try again.',
```

```
  });
});
```

Typed fakes exercise URL, valid external data and every malformed/HTTP/network branch offline.

__tests__/App.test.tsx

КОД · TSX

```tsx
import { act, fireEvent, render, waitFor } from '@testing-library/react-native';
import App from '../App';
import type { FetchImplementation, FetchResponse } from '../src/meals';

const mealPayload = {
  meals: [
    {
      idMeal: '52772',
      strMeal: 'Teriyaki Chicken',
      strMealThumb: 'https://img/chicken.jpg',
    },
  ],
};

function createResponse(payload: unknown, ok = true): FetchResponse {
  return { ok, json: async () => payload };
}

test('shows a MealDB result and rejects blank text before fetch', async () => {
  const fetchImpl: FetchImplementation = jest
    .fn<Promise<FetchResponse>, [string]>()
    .mockResolvedValue(createResponse(mealPayload));
  const ui = render(<App fetchImpl={fetchImpl} />);

  fireEvent.press(ui.getByRole('button', { name: 'SEARCH RECIPES' }));
  expect(ui.getByLabelText('Recipe error')).toHaveTextContent('Enter one ingredient.');
  expect(fetchImpl).not.toHaveBeenCalled();

  fireEvent.changeText(ui.getByLabelText('Ingredient'), 'chicken');
  fireEvent.press(ui.getByRole('button', { name: 'SEARCH RECIPES' }));

  await waitFor(() => expect(ui.getByText('Teriyaki Chicken')).toBeOnTheScreen());
  expect(ui.getByRole('image', { name: 'Photo of Teriyaki Chicken' })).toBeOnTheScreen();
});

test('shows no-results and retryable network error states', async () => {
  const fetchImpl: FetchImplementation = jest
    .fn<Promise<FetchResponse>, [string]>()
    .mockResolvedValueOnce(createResponse({ meals: null }))
    .mockRejectedValueOnce(new Error('offline'));
  const ui = render(<App fetchImpl={fetchImpl} />);

  fireEvent.changeText(ui.getByLabelText('Ingredient'), 'unknown');
  fireEvent.press(ui.getByRole('button', { name: 'SEARCH RECIPES' }));
  await waitFor(() => expect(ui.getByLabelText('Recipe empty')).toHaveTextContent('No recipes
found.'));
  expect(ui.getByLabelText('Recipe empty')).toHaveProp('accessibilityLiveRegion', 'polite');

  fireEvent.press(ui.getByRole('button', { name: 'SEARCH RECIPES' }));
  await waitFor(() => {
    expect(ui.getByRole('alert', { name: 'Recipe error' })).toHaveProp(
      'accessibilityLiveRegion',
      'assertive',
    );
  });
  expect(ui.getByRole('button', { name: 'RETRY SEARCH' })).toBeOnTheScreen();
});
```

```
test('starting a new search clears the previous result and announces loading', async () => {
  let resolveSecondRequest: ((response: FetchResponse) => void) | undefined;
  const secondRequest = new Promise<FetchResponse>((resolve) => {
    resolveSecondRequest = resolve;
  });
  const fetchImpl: FetchImplementation = jest
    .fn<Promise<FetchResponse>, [string]>()
    .mockResolvedValueOnce(createResponse(mealPayload))
    .mockReturnValueOnce(secondRequest);
  const ui = render(<App fetchImpl={fetchImpl} />);

  fireEvent.changeText(ui.getByLabelText('Ingredient'), 'chicken');
  fireEvent.press(ui.getByRole('button', { name: 'SEARCH RECIPES' }));
  await waitFor(() => expect(ui.getByText('Teriyaki Chicken')).toBeOnTheScreen());

  fireEvent.changeText(ui.getByLabelText('Ingredient'), 'beef');
  fireEvent.press(ui.getByRole('button', { name: 'SEARCH RECIPES' }));

  expect(ui.queryByText('Teriyaki Chicken')).not.toBeOnTheScreen();
  expect(ui.getByText('Loading recipes…')).toHaveProp('accessibilityLiveRegion', 'polite');

  await act(async () => {
    resolveSecondRequest?.(createResponse({ meals: null }));
  });
});

test('a failed search does not restore a result from the previous ingredient', async () => {
  const fetchImpl: FetchImplementation = jest
    .fn<Promise<FetchResponse>, [string]>()
    .mockResolvedValueOnce(createResponse(mealPayload))
    .mockRejectedValueOnce(new Error('offline'));
  const ui = render(<App fetchImpl={fetchImpl} />);

  fireEvent.changeText(ui.getByLabelText('Ingredient'), 'chicken');
  fireEvent.press(ui.getByRole('button', { name: 'SEARCH RECIPES' }));
  await waitFor(() => expect(ui.getByText('Teriyaki Chicken')).toBeOnTheScreen());

  fireEvent.changeText(ui.getByLabelText('Ingredient'), 'beef');
  fireEvent.press(ui.getByRole('button', { name: 'SEARCH RECIPES' }));
  await waitFor(() => expect(ui.getByRole('button', { name: 'RETRY SEARCH' })).toBeOnTheScreen());

  expect(ui.queryByText('Teriyaki Chicken')).not.toBeOnTheScreen();
});
```

The integration tests prove blank/success/empty/error/loading/stale-result behaviour with a real App and injected boundary.

Запуск и тесты

КОД · BASH

```
npm ci
npm run typecheck
npm test -- --runInBand
npx expo-doctor
npx expo export --platform web --output-dir dist-web
```

Ручная проверка

Проверьте empty input, `chicken`, loading, an unknown ingredient, offline Retry и `chicken breast` URL encoding.

Типичные ошибки

Не применяйте `any` к JSON, не считайте `meals: null` ошибкой, не сохраняйте stale results и не используйте search endpoint вместо filter endpoint.

Вопросы и ответы

Почему unknown? Сервер не гарантирует форму. **Почему injected fetch?** Тесты не зависят от сети. **Почему searched?** Empty state не виден до первого запроса.

Защита

Покажите `unknown → guards → Meal`, URL encoding, union narrowing и пять UI states.

Чек-лист сдачи

- ☐ Filter API, FlatList, name и thumbnail готовы.
- ☐ Blank/loading/empty/error/retry branches показаны.
- ☐ Strict TS, injected fetch и guards покрыты Jest.
- ☐ Нет ключа/API secrets/generated output; команды проверки выполнены.